

A Scalable Query-Aware Enormous Database Generator for Database Evaluation

Qingshuai Wang^{ID}, Yuming Li, Rong Zhang^{ID}, Ke Shu, Zhenjie Zhang, and Aoying Zhou

Abstract—Query-aware synthetic data generation is an essential and highly challenging task, important for database management system (DBMS) testing, database application testing and application-driven benchmarking. Prior studies on query-aware data generation suffer common problems of limited parallelization, poor scalability, and excessive memory consumption, making these systems unsatisfactory to terabyte scale data generation. In order to fill the gap between the existing data generation techniques and the emerging demands of enormous query-aware test databases, we design and implement a new data generator, called *Touchstone*. *Touchstone* adopts the random sampling algorithm instantiating query parameters and the new data generation schema generating the test database, to achieve fully parallel data generation, linear scalability and austere memory consumption. It has full support of outer joins as well as non-equi-joins for application-oriented data generation. Our experimental results show that *Touchstone* consistently outperforms the state-of-the-art solution on TPC-H workload by a 1000× speedup without sacrificing simulation fidelity.

Index Terms—Query-aware data generator, OLAP database testing, query generator

1 INTRODUCTION

QUERY-AWARE test databases are essential to database testings for analytical applications, helping database developers and users to launch effective evaluation. The application scenarios of query-aware data generators include DBMS testing, database application testing and application-driven benchmarking, e.g., [5], [20], [24]. Schema-aware data generators, e.g., [4], [23], generate test databases independent of the test queries, which only consider the data distribution of inter- and intra-attribute. Query-aware data generators for functional testing, e.g., [16], [24], [27], generate databases satisfying specific targets, e.g., *predicate coverage*. They fail to guarantee the similar workload characteristics of the test queries, therefore it's difficult to match the overheads of the query execution engine for real world workloads. A number of other studies, e.g., [5], [20], attempt to build query-aware data generators for performance evaluation. But the solution remains far from satisfactory, due to the lack of parallelization, scalability and memory usage control, as well as the narrow support to some kind of workload, e.g., non-equi-join.

In order to generate enormous query-aware test databases for performance evaluation, we design and implement *Touchstone* based on the following design principles: first, it should provide a *Full Parallelization* to support storage and access services for terabyte or even petabyte scale of data; second, it has a *Linear Scalability* to multiple nodes meeting the requirements of generating large test databases; third, it should guarantee *Austere Memory Consumption* to avoid meeting a generation bottleneck in maintaining dependencies relationships among columns. We have a close target with work [5], [6], [19], [20], but *Touchstone* tackles with the key insufficiencies among these studies. There are extremely complicated dependencies among columns, which are caused by the complex workload characteristics specified on the target test queries. Decoupling the generation of columns, especially for primary-foreign-key pairs, has been a challenging task, which decides the ability of parallelization. However, all of these approaches do not support fully parallel data generation because of the maintenance of huge shared intermediate status corresponding to the dependencies. Moreover, their memory consumptions strongly depend on the size of generation outputs. Once the memory is insufficient to host the intermediate status, I/O operations will significantly decrease the generation efficiency. The other key advantage of *Touchstone* is the support of non-equi-join and outer-join workloads satisfying the required workload characteristics, which are important for real world applications but not supported by the existing approaches. Last but not least, we can guarantee to generate one database instance (DBI) for multiple queries of the application, which is different from the approaches in [6], [19], [20].

There are two core techniques employed by *Touchstone* beneath the accomplishments of all above enticing features. First, *Touchstone* employs a completely new query instantiation scheme adopting the random sampling algorithm, which supports a large and useful class of queries, as well

- Qingshuai Wang, Rong Zhang, and Aoying Zhou are with the School of Data Science and Engineering, Shanghai Key Laboratory of Trustworthy Computing, East China Normal University, Shanghai 200062, China. E-mail: qswang@stu.ecnu.edu.cn, {rzhang, ayzhou}@dase.ecnu.edu.cn.
- Yuming Li is with Hua Wei Ltd, Shanghai 201200, China. E-mail: liyuming@stu.ecnu.edu.cn.
- Ke Shu is with PingCAP Ltd, Beijing 100192, China. E-mail: shuke@pingcap.com.
- Zhenjie Zhang is with Neuron Mobility Pte. Ltd., Singapore 118229. E-mail: zhenjie.zhang@neuron.sg.

Manuscript received 23 May 2020; revised 31 Mar. 2021; accepted 7 Feb. 2022.

Date of publication 23 Feb. 2022; date of current version 3 Apr. 2023.

This work was supported by the National Science Foundation of China under Grant 62072179, ECNU-TiDB Joint Project and 2020 the Key Software Adaptation and Verification Project (Database).

(Corresponding author: Rong Zhang.)

Recommended for acceptance by L. Zou.

Digital Object Identifier no. 10.1109/TKDE.2022.3153651

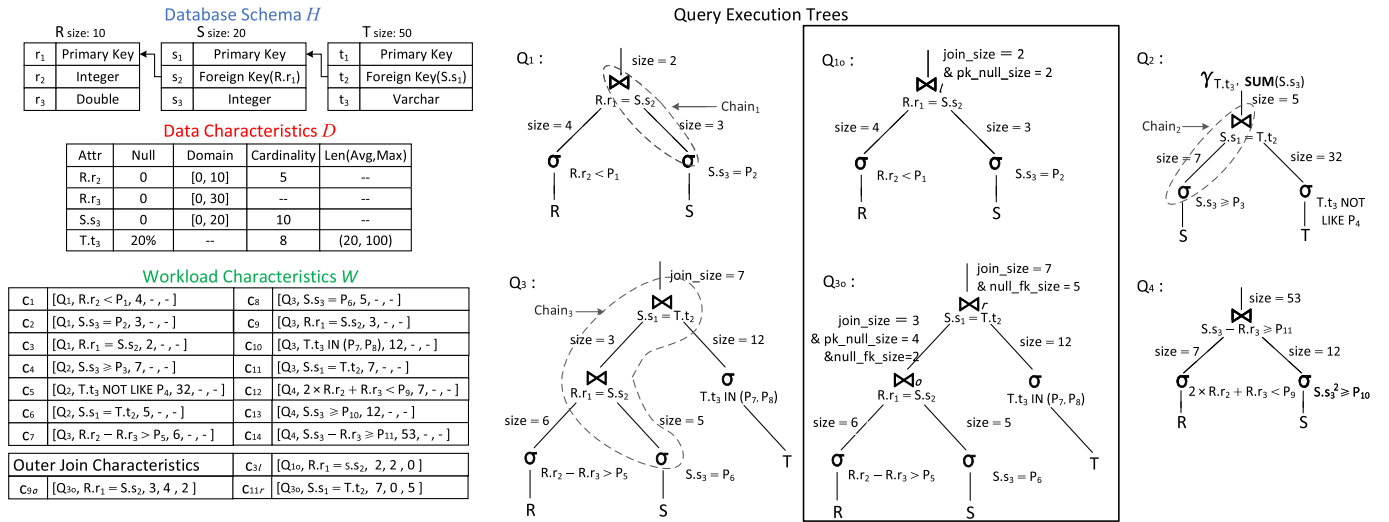


Fig. 1. Example inputs of database schema H , data characteristics D and workload characteristics W to *Touchstone*.

as the actual query execution trees from the running database system. Second, *Touchstone* is equipped with a new data generation scheme using the constraint chain data structure, which easily enables thread-level parallelization. This scheme allows *Touchstone* to easily scale up to dozens of nodes and hundreds of cores for parallelized database generation. In summary, *Touchstone* is a scalable query-aware data generator having a wide support to complex queries with specifications to workload characteristics of analytical applications, and achieves a 1000 $\times$ performance improvement against the state-of-the-art work MyBenchmark [20].

This paper is a significant extension of our conference version [18]. We extend the approach to 1) support outer-join operators, 2) add a comprehensive study about the exception processing and ratio adjustment, 3) take a new benchmark to verify our design, 4) supply theoretical analysis for *Touchstone*, and 5) provide more details of design and in-depth analysis of the results.

2 PRELIMINARIES

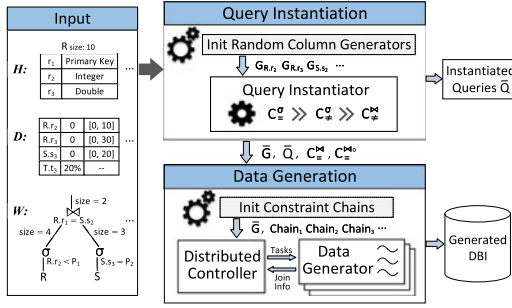
2.1 Problem Formulation

The input of *Touchstone* includes database schema H , data characteristics D and workload characteristics W , as illustrated in Fig. 1. H defines data types of columns, primary key and foreign key constraints. For example in Fig. 1, table S has 20 tuples and three columns, including a primary key $S.s_1$, a foreign key $S.s_2$ referencing to $R.r_1$ from table R , and an integer typed column $S.s_3$. Data characteristics D of columns are defined in a meta table, in which it defines the percentage of *Null* values, column domains, the cardinality of unique values and the average/maximum lengths for varchar typed columns. In our example, the user expects to see 8 unique strings with an average length of 20 and a maximum length of 100 for column $T.t_3$ with 20% *Null* values. Workload characteristics W are represented by a set of parameterized queries which are annotated with cardinality constraints. In Fig. 1, our sample input consists of six parameterized queries, i.e., $Q = \{Q_{1-4}, Q_{10}, Q_{30}\}$. The first four queries Q_{1-4} contain 11 variable parameters, i.e., $P = \{P_1, P_2, ..., P_{11}\}$. Each filter/join operator in the queries is

associated with a size constraint, defining the expected cardinality of the processing outcomes. If we zoom into Q_1 , there are three cardinality constraints, two of which have variable parameters, i.e., $R.r_2 < P_1$ and $S.s_3 = P_2$ with parameters P_1 and P_2 respectively. The last two queries Q_{1o} and Q_{3o} are the variants of Q_1 and Q_3 , in which inner joins are replaced by three kinds of outer joins, i.e., *left-outer* join, *full-outer* join, and *right-outer* join, with the cardinality constraints as $\{c_{3l}, c_{9o}, c_{11r}\}$. There are totally 17 filter/join operators with corresponding 17 cardinality constraints in our example, i.e., $C = \{c_{1-14}, c_{3l}, c_{9o}, c_{11r}\}$. Our target is to generate the three tables and instantiate all the variable query parameters. Definition of cardinality constraints is formulated in Definition 1.

Definition 1 Cardinality Constraint. Given a filter (σ) or join ($\bowtie$) operator, a cardinality constraint c is denoted as a quintuple $c = [Q, p, s, pk_null_size, null_fk_size]$, where Q indicates the involving query, p gives the predicate on the incoming tuples, and s is the expected cardinality of operator outcomes, and $pk_null_size/null_fk_size$ is the expected cardinality of nulls on the side of foreign/primary key for outer joins, if any.

The cardinality constraint c_1 in Fig. 1 is written as $[Q_1, R.r_2 < P_1, 4, -, -]$, indicating that the operator with predicate $R.r_2 < P_1$ in query Q_1 is expected to output 4 tuples. c_1 is not an outer join, and then there are no values set in the last two items, represented by ‘-’. While c_{90} is an outer join cardinality constraint, i.e., $[Q_{30}, R.r_1 = S.s_2, 3, 4, 2]$, indicating that the full outer join with predicate $R.r_1 = S.s_2$ in Q_{30} is expected to output 3 inner-join-tuples, and 4 (resp. 2) tuples with *nulls* in the field of $S.s_2$ (resp. $R.r_1$). The last two items are only valid for outer joins, so in the following explanation, we display the first three values for all non-outer-join constraints. These cardinality constraints generally characterize the computational workload of query processing engines, because the computational overhead mainly depends on the size of the data in processing. For conjunctive and disjunctive operators, their cardinality constraints can be split to multiple cardinality constraints for each basic predicate using the standard probability theory.

Fig. 2. The overall architecture of *Touchstone*.

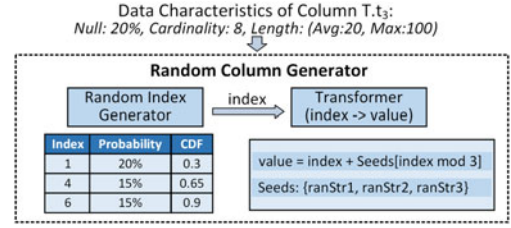
Though the focus of cardinality constraints is on filter and join operators, *Touchstone* also supports complex queries with other operators, including aggregation, groupby and orderby. For example, Q_2 in Fig. 1 applies groupby operator on $T.t_3$ and summation on $S.s_3$ over the grouped tuples. The cardinality of the output tuples from these operators, however, is mostly determined, if it does not contain a having clause. And such operators are usually engaged on the top of query execution tree, hence the output result cardinalities generally do not affect the total computational cost of query processing. Based on these observations, it is unnecessary to pose explicit cardinality constraints over these operators in *Touchstone* as in work [5], [19].

Based on the target operators and the predicates with equality or non-equality conditional expressions, we divide cardinality constraints into five types, i.e., $C = C_{=}^{\sigma} \cup C_{\neq}^{\sigma} \cup C_{\leq}^{\sigma} \cup C_{\geq}^{\sigma} \cup C_{\neq}^{\sigma}$. Accordingly, we classify the example constraints in Fig. 1 as $C_{=}^{\sigma} = \{c_2, c_5, c_8, c_{10}\}^1$, $C_{\neq}^{\sigma} = \{c_1, c_4, c_7, c_{12}, c_{13}\}$, $C_{\leq}^{\sigma} = \{c_3, c_6, c_9, c_{11}\}$, $C_{\geq}^{\sigma} = \{c_{14}\}$ and $C_{\neq}^{\sigma} = \{c_{3l}, c_{9o}, c_{11r}\}$. Following the common practice in [5], [29], [30], the equi-join operator is *always* applied on the pair of primary and foreign keys. Then we formulate the problem of query-aware data generation in Definition 2.

Definition 2 Query-Aware Data Generation Problem.

Given the input database schema H , data characteristic D and workload characteristics W , the objective of data generation is to generate a database instance (DBI) and instantiated queries, such that 1) the data in the tables strictly follows the specified data characteristics D ; 2) the variable parameters in the queries are appropriately instantiated; and 3) the executions of the instantiated queries on the generated DBI produce exactly the expected output cardinality specified by workload characteristics W on each operator.

While the general solution to query-aware data generation problem is NP-hard [25], we aim to design a data generator, by relaxing the third target in Definition 2. Specifically, the output DBI is expected to perform as closely as the cardinality constraints in C . Given the actual/expected cardinalities of processing outputs, i.e., $\{\hat{s}_1, \hat{s}_2, \dots, \hat{s}_n\}$, corresponding to constraints on the queries in $C = \{c_1, c_2, \dots, c_n\}$, we aim to minimize the global relative error $\frac{\sum_{c_i \in C} |c_i.s - \hat{s}_i|}{\sum_{c_i \in C} c_i.s}$. Notice that we can use other optimization metric, e.g., the sum of relative errors on each query or operator. Even if the user specified

Fig. 3. An example generator for column $T.t_3$.

workload in W contains conflicted constraints, *Touchstone* still attempts to generate a DBI with the best effort.

2.2 Overview of Touchstone

The input to *Touchstone* contains database schema H , data characteristics D of non-key columns and workload characteristics W . The output of *Touchstone* consists of two parts, namely the instantiated queries and the generated DBI. To support the generation procedure, the infrastructure of *Touchstone* is divided into two major components which are responsible for query instantiation and data generation respectively, as shown in Fig. 2.

Query Instantiation. Given the inputs H and D , *Touchstone* first initializes a group of random column generators for non-key columns, denoted by G . Each G_i corresponds to a column of the target tables. Given workload characteristics W , *Touchstone* instantiates the parameterized queries by adjusting the related column generators if necessary and choosing appropriate values for the variable parameters in the predicates of $c \in C_{=}^{\sigma} \cup C_{\neq}^{\sigma} \cup C_{\leq}^{\sigma}$. The instantiated queries $\bar{Q}$ are output to users, while the queries $\bar{Q}$ and the adjusted column generators $\bar{G}$ are fed into the data generation component with the details available in Section 3.

Data Generation. Given the instantiated queries $\bar{Q}$, and constraints over the equi-join operators $C_{=}^{\sigma}$ and outer join operators $C_{\neq}^{\sigma}$ specified in W , *Touchstone* decomposes the query trees annotated with constraints into constraint chains, in order to decouple the dependencies among columns, especially for primary-foreign-key pairs. The adjusted column generators $\bar{G}$ and constraint chains are distributed to all data generators for independent and parallel tuple generation. Technical details are available in Section 4.

2.3 Random Column Generator

We first initialize a group of random column generators $G = \{G_1, G_2, \dots, G_n\}$, which determine the data distributions of all non-key columns. A random column generator G_i in G is capable of generating values for the specified column, while meeting the required data characteristics in expectation.

Each G_i contains two parts, a random index generator and a value transformer as shown in Fig. 3. In the random index generator, the output index domain is the integers from 0 to $n-1$ while n is the specified cardinality of unique values in the corresponding column. Given an index, the transformer deterministically maps it to a value in the specified domain of the column and we adopt different transformers based on the type of the column. Taking the string type $T.t_3$ in Fig. 3 as an example, there are some seed strings pre-generated randomly, which satisfy the specified length

1. Constraints with $\{=, \neq, in, not in, like, not like\}$ are classified to $C_{=}^{\sigma}$.

requirements. We first select a seed string based on the input index and then concatenate the index and the selected seed string as the output value.

To manipulate the distribution of column values, there is a probability table in the index generator, which consists of a number of entries ordered by k_i . Each entry (k_i, p_i, c_i) specifies an index k_i , the probability p_i of occurrence, and the cumulative probability c_i for all indexes no larger than k_i . For saving memory usage, we compress the table by keeping only the entries with non-uniform probabilities. If an index does not appear in the probability table, its probability is automatically set by the uniform probability. In Fig. 3, we present an example column generator designed for column $T.t_3$, which contains 8 unique strings with average length 20 and maximum length 100. In the result generator, based on the indexes in $[0,7]$ generated by the index generator, the transformer outputs random strings with desired lengths, at probabilities $\{p_{0,2,3,5,7} = 0.1, p_1 = 0.2, p_{4,6} = 0.15\}$. Technical details are available in Section 3.

Value Generation. Given the column generator, first, a *Null* value is output with the probability of the specified percentage. If *Null* value is not chosen, the index generator randomly chooses a real number x in $(0, 1]$. By running binary search over CDF (c_i) in the probability table, the index generator retrieves an entry (k_i, p_i, c_i) that its c_i covers the random number x , i.e., $c_{i-1} < x \leq c_i$ (or $x \leq c_i$ when $i = 0$). If $x > c_i - p_i$, the index k_i is fed into the transformer, or else the index generator randomly chooses an index in the range $[k_{i-1} + 1, k_i - 1]$ (or $[0, k_i - 1]$ when $i = 0$) for the transformer. When the index generator fails to locate such an entry in the table, it picks up an index in $[k_m, n - 1]$, where k_m is the largest index in the probability table ($k_m = 0$ when there is no entry) and n is the cardinality of unique values specified on the column. With the input index, the transformer outputs its corresponding value.

3 QUERY INSTANTIATION

There are two general objectives in query instantiation, targeting to 1) construct the column generators for each non-key column in the tables; and 2) find concrete values for the variable parameters in the queries.

The query instantiation component is responsible for handling three types of constraints, i.e., $C_{=}^{\sigma}$, $C_{\neq}^{\sigma}$ and $C_{\neq}^{\infty}$. Note that the types of constraints $C_{=}^{\sigma}$ and $C_{=}^{\infty}$ involve matching between primary and foreign keys, which are tackled by the data generation process at runtime. In Algorithm 1, we list the general workflow of query instantiation. The algorithm iteratively adjusts the data distribution adopted by the column generators and the concrete values of the variable parameters, in order to meet the constraints as much as possible. The distribution adjustment on the column generator is accomplished by inserting entries in its probability table. In each iteration, the algorithm first re-initializes the column generators (line 3) by uniform distributions so that there is no entry in the probability table. It then adjusts the generators in $\bar{G}$ and instantiates the variable parameters in $\bar{Q}$ based on the equality constraints over filters (lines 4-7). It follows to revise the variable parameters in the queries in order to meet the non-equality constraints on filter and join operators (lines 8-14). The algorithm

outputs the new (adjusted) generators $\bar{G}$ and the instantiated queries $\bar{Q}$, when the global relative error for all constraints is within the specified threshold θ or the number of iterations reaches its maximum I . In this section, we discuss the processing strategies for these three types of constraints respectively.

3.1 Filters With Equality Constraint

Given all these equality constraints on the filter operators, i.e., $C_{=}^{\sigma}$, *Touchstone* groups the constraints according to the involved columns. In Fig. 1, there are four such kinds of constraints $C_{=}^{\sigma} = \{c_2, c_5, c_8, c_{10}\}$, among which, c_2 and c_8 target column $S.s_3$, and c_5 and c_{10} target column $T.t_3$. Note that all relational operators in equality constraints are handled by treating them as '='. For example, in Fig. 1, $c_5 = [Q_2, T.t_3 \text{ NOT LIKE } P_4, 32] \Rightarrow [Q_2, T.t_3 \text{ LIKE } P_4, 8] \Rightarrow [Q_2, T.t_3 = P_4, 8]$, where 20% is the percentage of *Null* value in column $T.t_3$.

Algorithm 1. Query Instantiation

Input: Initial generators G , input queries Q , error threshold θ and maximal iterations I
Output: New generators $\bar{G}$ and instantiated queries $\bar{Q}$

- 1: Initialize $\bar{Q} \leftarrow Q$
- 2: **for all** iteration $i = 1$ to I **do**
- 3: Initialize $\bar{G} \leftarrow G$
- 4: **for all** constraint $c \in C_{=}^{\sigma}$ **do**
- 5: Adjust the generator in $\bar{G}$ for the column within c
- 6: Instantiate the corresponding parameter in $\bar{Q}$
- 7: **end for**
- 8: **for all** $c \in C_{\neq}^{\sigma}$ **do**
- 9: Instantiate the corresponding parameter in $\bar{Q}$
- 10: **end for**
- 11: **for all** $c \in C_{\neq}^{\infty}$ **do**
- 12: Obtain constraints from all descendant nodes
- 13: Instantiate the corresponding parameter in $\bar{Q}$
- 14: **end for**
- 15: Calculate the global relative error e
- 16: **if** $e \leq \theta$ **then**
- 17: **return** $\bar{G}$ and $\bar{Q}$
- 18: **end if**
- 19: **end for**
- 20: **return** $\bar{G}$ and $\bar{Q}$ (historical best solution with minimum e)

The processing of $C_{=}^{\sigma}$ includes three steps. First, it randomly selects an index and obtains the corresponding value from the transformer of the column generator for instantiating each variable parameter in the equality constraints. Second, it updates the occurrence probability of the selected index by inserting an entry in the probability table, in order to meet the required intermediate result cardinality. Whether the filter is the leaf node of the query execution tree or not, the probability of the inserted entry is calculated as $\frac{s_{out}}{s_{in}}$, where s_{in} (resp. s_{out}) is the size of input (resp. output) tuples. Then it calculates the cumulative probabilities in the probability table of the adjusted generator. For example in Fig. 3, there are three entries inserted for p_4 , p_7 and p_8 on $T.t_3$. Except 20% *nulls*, it has 40 ($=50 \times 0.8$) valid values. The entry with index 1 is inserted for instantiating p_4 in c_5 with $p_1 = 1 - \frac{32}{40} = 0.2$, while indexes 4 and 6 are inserted for p_7 and p_8 in c_{10} with $p_4 = p_6 = \frac{1}{2} \times \frac{12}{40} = 0.15$. For the cardinality is 8 for $T.t_3$, the remaining 5 values are uniformly distributed and

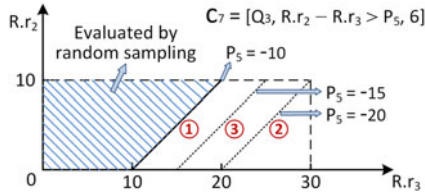


Fig. 4. Given the predicate in c_7 in Fig. 1, we attempt to cut the space by revising parameter P_5 , e.g., 1→2→3. For a concrete P_5 , random sampling is used to estimate the number of tuples meeting the predicate. The value is returned for P_5 , after reaching the desired precision or the maximum iterations.

each has the probability $0.1 (= \frac{1-0.2-0.15*2}{5})$, so the cumulative probability of index 1 is $0.3 (= 0.1 + 0.2)$.

Suppose there are k variable parameters in the constraints of $C_{\neq}^{\sigma}$. The total complexity of the processing is $O(k \log k)$, because the algorithm only needs to instantiate the parameters one by one, and inserts an entry into the probability table in order of selected index for each parameter instantiation.

3.2 Filters With Non-Equality Constraint

Filters with Non-Equality Constraint, i.e., $C_{\neq}^{\sigma}$, could involve one non-key column, e.g., c_1 or multiple non-key columns, e.g., c_7 in Fig. 1. Our strategy handling $C_{\neq}^{\sigma}$ is to find the concrete parameters generating the best matching output cardinalities against the constraints, based on the data distributions adopted by the column generators.

Since the cardinality of tuples satisfying the constraints is monotonic with the growth of the variable parameters, it suffices to run a binary search over the parameter domain to find the concrete values for the variable parameters. For example in Fig. 4, the cutting line represents the parameter in c_7 , which decides the ratio of tuples in the shadow area, i.e., satisfying the constraint. By moving the cut line, the likelihood of tuples in the shadow area changes correspondingly. The challenge here is how to estimate the probability of tuples falling in the shadow area. To tackle the problem, we adopt the random sampling algorithm. Specifically, we sample a batch of values from involved columns according to the probability table and compute the probability that the constraint is satisfied. Note that the binary search may not be able to find a parameter with the desired precision, based on the determined data distribution of columns after processing constraints in $C_{=}$. For example in Fig. 3, if we have a constraint $[Q_m, T.t_3 < p_n, 0.2]$, there will be no valid value for p_n , because index 1 covers the CDF from 10% to 30%. Therefore, in Algorithm 1, we try to instantiate the parameters in $C_{\neq}^{\sigma}$ upon different data distributions by iteration. Since the output sizes in equality constraints over filters are usually far less than table sizes, the processing of $C_{=}$ does not make the data distributions of column values extremely skewed. Therefore, Algorithm 1 can often complete query instantiation by a limited number of iterations.

The complexity of the approach is the product of the number of iterations in parameter value search and the computational cost of the random sampling in each iteration. The number of iterations for binary search is logarithmic to the domain size of the parameter, decided by the minimal and maximal values that the expression with

multiple columns could reach. The cost of random sampling depends on the complexity of the predicate, which usually only involves a few columns.

3.3 Joins With Non-Equality Constraint

Processing joins in $C_{\neq}^{\sigma}$ is slightly different from the filters in $C_{\neq}^{\sigma}$, because the columns involved in join predicates may overlap with the columns in the predicates of their child nodes as Q_4 in Fig. 1, which usually does not happen to filters in the query execution tree. Therefore, we must process the constraints in a bottom-up manner without the premise of probability independence, such that the precedent operators are settled before the join operator with a non-equality constraint is handled. For Q_4 , after *Touchstone* concretizes the parameters P_9 and P_{10} in constraints c_{12} and c_{13} , the input data to the join operator with constraint c_{14} is determined. Based on the characteristics of the inputs, we apply the same binary search strategy designed for filter operators to locate the optimal parameter, e.g., P_{11} for the desired result cardinality. Since the algorithm is identical to that for filter operator, we hereby skip detailed algorithm descriptions as well as the complexity analysis.

4 DATA GENERATION

Given the generators of all non-key columns and the instantiated queries, the data generation component in Fig. 2 is responsible for assembling tuples based on the outputs of the column generators. The key technical challenge here is to meet the equality constraints over the join operators, i.e., $C_{=}$ and $C_{=, \neq}^{\sigma}$, which involve the dependencies among primary and foreign keys from multiple tables. To tackle the problem, we design a new tuple generation schema, which focuses on the manipulation of foreign keys only.

Tuple generation consists of two steps in *Touchstone*. In the first *compilation* step, it orders the tables as a generation sequence and decomposes the query trees into constraint chains for each target table. In the second *assembling* step, the working threads independently generate tuples for the tables based on the order from the compilation step. For each tuple, the working thread fills values in the columns by calling the column generators independently and incrementally assigns a primary key, while leaving the foreign keys blank. By iterating the constraint chains associated with the table, we identify the appropriate candidate keys for each foreign key based on the maintained join information of the referenced primary keys, and randomly assign one of the candidate keys to the tuple.

4.1 Tuple Generation With Inner Joins

4.1.1 Compilation

Since primary-foreign-key dependencies form a directed acyclic graph (DAG), *Touchstone* constructs a topological order over the tables, which decides the generation order of tables. In Fig. 5, we illustrate the generation order over three tables, $R \rightarrow S \rightarrow T$, based on the schema H in Fig. 1. In order to decouple the dependencies among columns and facilitate parallelizing, *Touchstone* decomposes the query trees annotated with equal join constraints into *constraint chains* with respect to each table, which have three types,

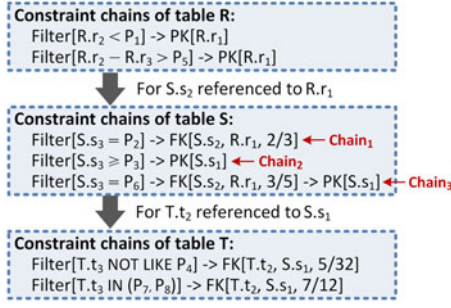


Fig. 5. Results of constraint chain decomposition.

namely FILTER, PK and FK. The constraint chains for a table are defined as the sequences of cardinality constraints over the operators with descendant relationships in the query trees. In Fig. 5, we present all the constraint chains for tables R , S and T . Table S has three constraint chains extracted from Q_1 , Q_2 and Q_3 considering $C_{\text{max}}^{\text{FK}}$, maked by dotted lines in Fig. 1. Note that there is no constraint chain from Q_4 which has no equi-join operator.

Algorithm 2. Tuple Generation

Input: Column generators $\bar{G}$, constraint chains of the target table Ω , join information tables of referenced primary key and current primary key t_{rpk} and t_{pk}

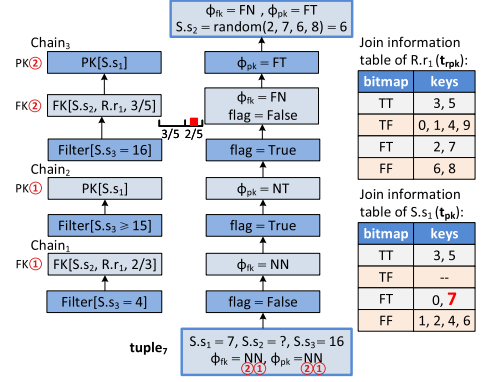
Output: Tuple r and join information table t_{pk}

```

1:  $r.pk \leftarrow$  a value assigned incrementally
2:  $r.columns \leftarrow$  values output by column generators  $\bar{G}$ 
3:  $\phi_{fk} \leftarrow N \dots N, \phi_{pk} \leftarrow N \dots N$ 
4: for all constraint chain  $\omega \in \Omega$  do
5:    $flag \leftarrow True$ 
6:   for all constraint  $c \in \omega$  do
7:     if ( $c$  is FILTER) && ( $c.predicate$  is False) then
8:        $flag \leftarrow False$ 
9:     else if  $c$  is PK then
10:       $\phi_{pk}[i] \leftarrow flag // i$  is the bit index for  $c$ 
11:    else if ( $c$  is FK) &&  $flag$  then
12:      //  $i$  is the bit index for  $c$ 
13:       $\phi_{fk}[i], flag \leftarrow (random[0, 1] < c.ratio)$ 
14:      if ( $c$  is RFK) then
15:         $flag \leftarrow True$ 
16:      end if
17:    end if
18:  end for
19: end for
20:  $r.fk \leftarrow$  a value selected from  $t_{rpk}$  satisfying  $\phi_{fk}$ 
21: Add  $r.pk$  in the entry of  $t_{pk}$  with bitmap  $\phi_{pk}$ 
22: return  $r$  and  $t_{pk}$ 

```

Each FILTER constraint keeps the predicate with the instantiated parameters. Each PK constraint in the chain records the column name of the primary key. Each FK constraint maintains a triplet, covering the names of the foreign key and the referenced primary key, and the expected ratio of tuples satisfying the predicate on $\bowtie_{\text{max}}$. The second constraint of $Chain_1$ for table S in Fig. 5, for example, is $FK[S.s2, R.r1, \frac{2}{3}]$, indicating the foreign key is $S.s2$, the referenced primary key is $R.r1$ and two out of three tuples in table S are expected to meet the predicate $S.s2 = R.r1$ while satisfying the previous filter predicate $S.s3 = P_2$. The expected ratios in FK constraints are calculated based on

Fig. 6. Running example of tuple generation for table S .

the cardinality requirements of the specified cardinality constraints.

4.1.2 Assembling

For simplicity, we assume that there is a single-column primary key and one foreign key in the table. Note that our algorithm can be naturally extended to handle tables with composite primary keys and multiple foreign keys with details in Appendix D, which can be found on the Computer Society Digital Library at <http://doi.ieeecomputersociety.org/10.1109/TKDE.2022.3153651>. The result constraint chains are distributed to all working threads on multiple nodes for parallel tuple generation. When generating tuples for a specified table, each working thread maintains two bitmap data structures at runtime, i.e., ϕ_{fk} and ϕ_{pk} . They are used to keep track of the status of joinability, e.g., whether the generating tuple satisfies individual predicates over $\bowtie_{\text{max}}$, for primary key and foreign key, respectively. The length of the bitmap ϕ_{fk} (resp. ϕ_{pk}) is equivalent to the number of FK (resp. PK) constraints in all chains of the target table. Each bit corresponds to a FK/PK constraint, which has three possible values, i.e., T , F and N , indicating if the join status is successful, unsuccessful or null. In Fig. 5, for example, table S has two FK constraints and two PK constraints, resulting in 2-bit representations for both ϕ_{fk} and ϕ_{pk} .

Based on the bitmap ϕ_{pk} , *Touchstone* constructs the join information table to track the status of joinability of primary keys. There are two attributes in the join information table, i.e., bitmap and keys, indicating the status of joinability and the corresponding satisfying primary key values. In Fig. 6, we show two join information tables of primary keys $R.r1$ and $S.s1$, which are constructed during the generation of table R and S , and used for generating the foreign key $S.s2$ of table S and $T.t2$ of table T . The join information table of $R.r1$ is maintained in the generation of table R , which is ready for generating the foreign key $S.s2$ of table S . During the generation of table S , the join information table of $S.s1$ is maintained for generating the foreign key $T.t2$ of table T . Note that the keys corresponding to an entry may be empty (such entries will not be stored in practice), which means there is no primary key satisfying the desired joinability status, and the details are discussed in Section 5.

The tuple generation algorithm is listed in Algorithm 2 with a running example shown in Fig. 6. A new tuple, i.e., $tuple_r$, for table S is initialized as $(S.s1 = 7, S.s2 = ?, S.s3 =$

16), $\phi_{fk} = NN$ and $\phi_{pk} = NN$ (lines 1-3). The *flag* is set to True before traversing each constraint chain (line 5), which is used to track if the predicates from the precedent constraints of current chain are fully met. On *Chain₁*, since the predicate in the first FILTER constraint is $S.s_3 = 4$, *flag* is then set to False (line 8), and algorithm does not need to handle the next FK constraint (line 11). On *Chain₂*, *tuple₇* satisfies the predicate $S.s_3 \geq 15$, resulting in the update of the bitmap representation as $\phi_{pk} = NT$ (line 10). On *Chain₃*, after passing the first FILTER constraint, the corresponding bit of next FK constraint in ϕ_{fk} is randomly flipped to *F* at the probability of $\frac{2}{5}$, because the expected *ratio* that satisfying the FK constraint is $\frac{3}{5}$. The *flag* is set to False (line 13) to reflect the failure of full matching of precedent constraints for the later PK constraint. Then, the bit for the next PK constraint in ϕ_{pk} is set as *F* according to the value of *flag* (line 10). Therefore, the two bitmaps are finalized as $\phi_{fk} = FN$ and $\phi_{pk} = FT$. Then the algorithm identifies two entries matching $\phi_{fk} = FN$, namely satisfying the *T/F* requirements on the corresponding bits of ϕ_{fk} , with bitmaps *FT* and *FF* respectively, in the join information table of *R.r₁*. Given these two entries, it randomly selects (line 20) a foreign key, e.g., 6, from four candidate referenced primary keys $\{2, 7, 6, 8\}$, which are all appropriate for foreign key $S.s_2$. Finally, the algorithm updates (line 21) the join information table of $S.s_1$ by adding the primary key $S.s_1 = 7$ into the entry with bitmap *FT*.

For a table, if there are k non-key columns, m constraints in the related constraint chains and n entries in the join information table of the referenced primary key, the complexity of tuple generation mainly consists of three parts: k times of calling random column generators for filling the values of non-key columns, the traversing over m constraints within chains for determining the joinability statuses of foreign keys and primary keys, and the comparing with n bitmaps in the join information table for searching the appropriate foreign key candidates. For practical workloads, k , m and n are all small numbers, e.g., $k \leq 12$, $m \leq 20$ and $n \leq 40$ for TPC-H [3] workload, so our tuple generation is highly efficient.

4.1.3 Compression for Join Information Table

$C_{\subseteq}^{\approx}$ represents the most popular dependency among tables. Since the relationship of foreign and primary keys can be many to one and the intermediate result cardinality is the main factor that affects query performance, we design a compression method by storing less primary key values in the join information table but still promise the randomness of remaining values.

Assuming the size of keys in an entry of join information table is N , which is hard to know in advance and may be very large. We aim to store only L ($L \ll N$) keys and promise the approximately uniform distribution of these L ones among all N values. The compression method is a variant of reservoir sampling implemented as follows: we store the first L arriving values in the keys, if any; and for the i^{th} ($i > L$) arriving value, we randomly replace a value stored previously for the entry with the probability of L/i . By such a method, the space complexity of the join information table is reduced to $O(n * L)$, where n is the number of entries and L is the maximum allowed size of keys in each entry. Since n is

generally small, e.g., $n \leq 40$ for TPC-H workload, and L can be set to thousands, the memory consumption and network transmission of the join information table are acceptable.

Proposition 1. For a key valid for Inner-Join, the probability that it is selected to be in L is $\frac{L}{N}$ if $N > L$, or else it is 1. N and L are the size of valid keys and the key size limitation for each entry.

Proof 1. Supposing i is the id of the arriving key, generated incrementally with $i \geq 1$. When $i \leq L$, there is still free space in the entry and the key must be stored, so the probability inserted into L is 1.

When $i > L$, for the incoming key, supposing its probability selected into L is L/i . The current key set for the entry is called L_i .

According to our compression method, the $(i + 1)^{th}$ value is selected into L with probability $L/(i + 1)$ by a uniform distribution. For any existed key $v \in L_i \cap L_{i+1}$, we calculate its occurrence probability as

$$P(v \in L_{i+1}) = P(v \in L_i) * (1 - P(\text{remove } v)) \\ = \frac{L}{i} (1 - \frac{L}{i+1} \frac{1}{L}) = \frac{L}{i} \frac{i}{i+1} = \frac{L}{i+1}.$$

In summary, for any positive integer N ($N \gg L$), the probability that a key exists in L is L/N . $\square$

4.2 Tuple Generation With Outer Joins

Previously, we tackle $C_{\subseteq}^{\approx}$ by using join information table to keep the join status of keys. Outer joins including left/right outer joins ($\bowtie_l/\bowtie_r$), and full outer joins ($\bowtie_o$). The main idea is to guarantee the size of *nulls* in the join results from either primary keys or foreign keys besides the cardinality requirement of $\bowtie_{\subseteq}$. For example, considering c_{90} with a full outer join constraint, 3 results are generated by the equi-join, 4 results by the left outer join (4 *nulls* from foreign keys in Table S), and 2 results by the right outer join (2 *nulls* from the keys in Table R). In order to explain our processing, we replace Q_1 by Q_{1o} with a left outer join, and Q_3 by Q_{3o} with a full outer join and a right outer join. Correspondingly, cardinality constraints in Q_{1o} include c_1 , c_2 and c_{3l} ; cardinality constraints in Q_{3o} include c_7 , c_8 , c_{9o} , c_{10} and c_{11r} . $\bowtie_o$ can be divided into a left outer join and a right outer join, so there are two left outer joins on table *R*, i.e., in c_{3l} and c_{9o} , two right outer joins on table *S* (c_{9o}) and *T* (c_{11r}). Notice that Q_{1o} and Q_{3o} have the same cardinality constraints in $C_{\subseteq}^{\approx}$ with Q_1 and Q_3 , and we will construct the same 2-bit-bitmap join information table T_0 in Fig. 7 as done with join information table *R* of Fig. 6.

4.2.1 Left Outer Join

In order to generate the required *pk_null_size* tuples, the foreign keys can only be joined with a limited number of primary keys. In other words, the *pk_null_size* primary keys cannot be populated into the foreign key column. So we need to generate a reduced join information table, e.g., T_3 in Fig. 7, for populating foreign keys. In order to reduce from T_0 to T_3 , we construct the *Filter Table* T_2 to tell the *filter probability* of each status, i.e., the percentage of primary keys that will not be populated to the foreign key column.

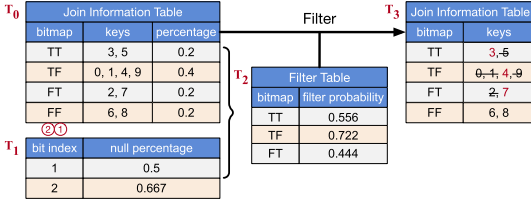


Fig. 7. Filter table for left outer join on Table R.

To obtain T_2 , we first calculate the key distributions among join statuses, i.e., *percentage* of T_0 in Fig. 7. T_1 records the *null* percentage for each join, i.e., *bit index*, corresponding to the *bitmap* for primary keys in T_0 . Let's take *bit index* = 2 for example. It is inserted for the full outer join in Q_{30} , i.e., c_{90} . In the query tree of Q_{30} , it outputs 6 tuples by σ operator in c_7 to the parent $\bowtie_o$ operator in c_{90} . The cardinality constraint in c_{90} is $[Q_{30}, R.r_1 = S.s_2, 3, 4, 2]$ which means it generates 3 results for the equi-join and 4 tuples for the left outer join from these 6 tuples. So the *null percentage* for *bit index* = 2 is 0.667 ($= 4/6$).

Based on T_1 , we learn that for the first left outer join in c_{30} , 50% of the valid keys from Table R (*bitmap* = TT and FT) join with *nulls*; and for the second left outer join, 66.7% of the keys (*bitmap* = TT and TF) join with *nulls*. Therefore, we need to filter the valid keys constructed for $\bowtie_{=}$ operators to satisfy the cardinality constraints for left outer joins in c_{3l} and c_{90} . Suppose M is the entry size of T_1 and the *null percentage* of the j^{th} *bit index* is β_j , $M = 2$ and $\beta_1 = 0.5$ in Fig. 7. All bitmaps in T_0 is V . V_j represents the subset of bitmaps with j^{th} bit set to T , e.g., $V_2 = \{TT, TF\}$. For a bitmap i ($i \in V_j$), α_i represents its percentage of keys, e.g., $\alpha_{TT} = 0.2$. For guaranteeing the *null percentages* specified in T_1 , each bitmap i in T_0 has a filter probability x_i , e.g., $x_{TT} = 0.556$ in T_2 . The total filter probability on V_j is β_j calculated by Eqn. (1)

$$\frac{\sum_{V_j} \alpha_i * x_i}{\sum_{V_j} \alpha_i} = \beta_j, \forall i \in V_j, x_i \in [0, 1], j \in [1, M]. \quad (1)$$

We take the quadratic optimization to find a set of x_i for V_j to minimize the distances of $\sum_{V_j} (x_i - \beta_j)$ in Eqn. (1). Different bitmaps occupy different percentages of keys, which are taken as the weights for optimization. Finally for all M *bit indexes* in T_1 , we formulate the optimization in Eqn. (2)

$$\min_{x_i, i \in V} \sum_{j=1}^M \sum_{i \in V_j} \frac{\alpha_i}{\sum_{V_j} \alpha_k} * (x_i - \beta_j)^2. \quad (2)$$

After we fill all *filter probabilities* in *Filter Table*, i.e., T_2 , we can filter the keys and get a new *Join Information Table*, i.e., T_3 , to guarantee the size of *null-mapping* tuples. For the example in Fig. 7, we calculate the *filter probability* for each bitmap in T_2 by optimizing

$$\begin{aligned} \min_{x_{TT}, x_{TF}, x_{FT}} & \frac{0.2 * (x_{TT} - 0.5)^2}{0.2 + 0.2} + \frac{0.2 * (x_{FT} - 0.5)^2}{0.2 + 0.2} \\ & + \frac{0.2 * (x_{TT} - 0.667)^2}{0.2 + 0.4} + \frac{0.4 * (x_{TF} - 0.667)^2}{0.2 + 0.4} \\ s.t. & \frac{0.2 * x_{TT} + 0.2 * x_{FT}}{0.2 + 0.2} = 0.5, \frac{0.2 * x_{TT} + 0.4 * x_{TF}}{0.2 + 0.4} = 0.667 \\ & 0 \leq x_{TT}, x_{TF}, x_{FT} \leq 1. \end{aligned}$$

Then we have $x_{TT} = 0.556$, $x_{TF} = 0.722$, and $x_{FT} = 0.444$. For $x_{TT} = 0.556$, we need filter $0.556 * 2 = 1.112 \approx 1$ tuple from the valid keys for *bitmap* = TT in T_0 . Since *filter probabilities* are used for keys valid for equi-joins, FF does not exist in *Filter Table*. Similarly, we filter 3 tuples for TF and 1 tuple for FT . Finally, we generate the new join information table T_3 , based on which we can guarantee to generate 2 *null-mapping* tuples for *bit index* = 1 and 4 *null-mapping* tuples for *bit index* = 2 required in c_{3l} and c_{90} .

If a bit in the *bitmap* represents the join status for the $\bowtie_{=}$ operator, it will have no requirement for filtering the keys considering the outer joins and we then compress the status with N in the *bitmap*. For example, if the *bitmap* in T_0 has three bits with the 3th one used by an inner join, we only aggregate the probabilities based the first two bits. So the percentage of keys for bitmaps TTT and FTT are aggregated as NTT . The *Filter Table* has the same size as T_2 , but the entries are NTT , NTF and NFT . After calculating the *filter probability* for each *bitmap* by Eqn. (2), e.g., NTT , we can simply assign the same *probability* to both TTT and FTT .

Based on the keys in the revised join information table, i.e., T_3 , *Touchstone* populates tuples by randomly selecting the keys and each key shall be used for at least once to guarantee the size of *nulls*.

4.2.2 Right Outer Join and Full Outer Join

For right outer joins, all of the items from the referencing table are output to the parent operator, if any, and the size of *null-mapping* tuples is essentially the size of tuples that cannot match the referenced primary keys by the referencing foreign keys. We use *RFK* to identify $\bowtie_r$ operator in line 14-16 in Algorithm 2 by setting *flag* = *True* to transmit all items from the referencing table. For example, in Q_{30} , we have a right outer join constraint included in c_{90} , that is a full outer join as in Fig. 1, the foreign key constraint $FK[S.s_2, R.r_1, 3/5]$ on $Chain_3$ in Fig. 6 is then marked as *RFK* so as to set *flag* = *True* in line 15. In the next operator $\bowtie_r$, we then have $\Phi_{pk} = TT$ for all the keys from Table S with *size* = 5 is sent directly to root $\bowtie_r$ operator. A full outer join, i.e., $\bowtie_o$, can be divided into a left and a right outer joins, which can be tackled with independently.

4.2.3 Maintenance of Join Information Table

In order to guarantee the size of *null-mapping* tuples specified by left outer join operators, if any, compression of join information table to size L is not valid any more. In such a case, valid keys for inner joins can be dumped into disk for persistent store, if the keys are too many to be kept in memory considering the specified size requirement, e.g., L . During the data generation, it uses all keys stored to populate the foreign keys.

5 HANDLING MISMATCH CASES

There are some joinability statuses of the primary keys that never occur because of the conditions in predicates. For example, in Fig. 6, if a tuple does not satisfy $Filter[S.s_3 \geq 15]$ in $Chain_2$, it must be false for $Filter[S.s_3 = 16]$ in $Chain_3$, so the *bitmap* ϕ_{pk} for primary key $S.s_1$ can not be TF . Therefore, in order to avoid generating the *bitmap* ϕ_{fk} that does

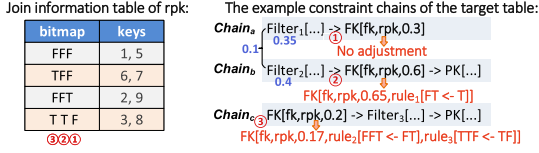


Fig. 8. An example of adjustments to FK constraints.

not have any matching entry in the join information table of the referenced primary key, we propose to add rules to manipulate relevant FK constraints.

Fig. 8 gives an example of adjustments to FK constraints for handling the mismatch case. There are three FK constraints with the serial numbers of 1, 2 and 3 in three constraint chains, respectively. Since there are four bitmaps that are not presented in the join information table of the referenced primary key r_{pk} corresponding to the foreign key f_k of the target table, i.e., FTT, TTT, TFT, FTF, three rules are added in the 2nd and 3rd FK constraints to avoid producing any ϕ_{f_k} triggering the mismatch case. For example, there is a rule $[FT \leftarrow T]$ added in the 2nd FK constraint, which indicates that the status of the 2nd FK constraint must be F if the status of the 1st FK constraint is T in the tuple generation. The general algorithm of adjustments to FK constraints and the corresponding analyses are presented in Section 5.1. Since there are extra F statuses forcibly generated by the added rule for the 2nd FK constraint, the actual ratio of tuples satisfying the corresponding predicate could be lower than the expected ratio 0.6. Consequently, it is necessary to adjust the expectation ratio in the 2nd FK constraint for eliminating the impact of the added rule. The adjustment strategy is discussed in Section 5.2. To reflect the adjustments to FK constraints, a minor modification is applied on the original tuple generation algorithm on line 13 in Algorithm 2. Specifically, the updated algorithm first checks all existing rules in current FK constraint. If there is a rule which can be applied to the statuses of previous constraints, ϕ_{f_k} and $flag$ are updated according to the rule. Otherwise, the algorithm updates ϕ_{f_k} and $flag$ by the probability based on the adjusted ratio.

For two rules, if they have the same conditions and the opposite target statuses, we call them conflicting rules. During the adjustment, there may be some conflicting rules added in the FK constraints. But the conflicting rules added in FK constraints would never be used as declared in Proposition 2. Detailed discussion can be found in Appendix A, available in the online supplemental material.

Proposition 2. *In the tuple generation, it would not have two conflicting rules applied at the same time for any FK constraint, and all conflicting rules in the FK constraints would never be applied.*

5.1 Adjustments to FK Constraints

In Algorithm 3, we list the general workflow of adjustments to FK constraints. For each foreign key in the table, we launch the adjustment based on the join information tables of referenced primary keys.

First, the algorithm initializes constraint chains $\bar{\Omega}$ from Ω in a random order (line 1). A poorly chosen order may not generate desired adjustment results. In the worst case, the

algorithm may need to attempt all possible orders of the constraint chains.

Algorithm 3. Adjustments to FK Constraints

Input: Constraint chains Ω and join information table of referenced primary key t_{rpk}

Output: Adjusted constraint chains $\bar{\Omega}$

```

1: for all  $\bar{\Omega}$  initialized from  $\Omega$  in a random order do
2:   Initialize  $C \leftarrow$  all FK constraints ordered as in  $\bar{\Omega}$ 
3:   for all index  $i = 1$  to  $C.size$  do
4:     for all possible joinability status  $\phi$  of  $c_1$  to  $c_i$  do
5:       if mismatch case is occurred over  $\phi$  and  $t_{rpk}$  then
6:         Add a rule built according to  $\phi$  into  $c_i$ 
7:       end if
8:     end for
9:     Adjust  $c_i.ratio$  based on added rules
10:  end for
11: Calculate error  $e$  caused by illegal adjustments
12: if  $e = 0$  then
13:   return  $\bar{\Omega}$ 
14: end if
15: end for
16: return  $\bar{\Omega}$  (historical best solution with minimum  $e$ )

```

Given a $\bar{\Omega}$ (line 2), the algorithm then traverses all FK constraints in order (line 3). For i^{th} FK constraint, i.e., c_i , it iterates over all possible joinability statuses considering constraints c_1 to c_i to identify joinability statuses that may generate mismatch cases (lines 4-8). For example, for a 3-bit bitmap, i.e., $\{s_3 s_2 s_1\}$, when it reaches the 3rd FK constraint, the algorithm iterates over 2^3 joinability statuses with $s_i \in \{T, F\}$. When a mismatch case is detected (line 5) over a joinability status ϕ , namely there is no satisfying referenced primary key in t_{rpk} , a rule is added into current FK constraint (line 6). For example, if the mismatch case occurs over $\phi = TFT$, a rule $[FFT \leftarrow FT]$ is generated for the 3rd FK constraint, indicating that its status s_3 must be F if the statuses of previous two FK constraints $s_2 s_1 = FT$. After iterating over all possible joinability statuses for i^{th} FK constraint, the algorithm adjusts its ratio based on the added rules to ensure the expected output cardinality (line 9). Next, when all FK constraints in $\bar{\Omega}$ are processed, it calculates the error caused by all illegal adjustments that the corresponding adjusted ratios are not between 0 and 1 (line 11). The detailed calculation of ratio adjustment and error is introduced in Section 5.2. Finally, it outputs the constraint chains $\bar{\Omega}$ with adjusted FK constraints, when there is no illegal adjustments or it has traversed all the orders of constraint chains.

For a table with one foreign key, supposing there are k FK constraints in all chains and m entries in the join information table of the referenced primary key, we calculate its complexity in Algorithm 3. We only need to consider the order of those k constraint chains. The complexity of the adjustment is $O(k! \cdot (\sum_{i=1}^k 2^i) \cdot m)$, where $k!$ is the number of all possible orders of constraint chains, $\sum_{i=1}^k 2^i$ is the sum of numbers of all possible joinability statuses for processing all FK constraints and m is the numbers of checking whether the mismatch case is occurred over a joinability status. For practical workloads, k and m are both small numbers, e.g., $k \leq 8$ and $m \leq 40$ for TPC-H workload. Note that the algorithm is

called only once before starting the tuple generation for the target table. Therefore, the cost of the adjustment is usually affordable. To save unnecessary computation, we could find a solution among the best top 1% ones by running a constant number of iterations and then $k!$ is replaced by a constant number.

5.2 Ratio Adjustment

In Algorithm 4, we present the way of ratio adjustment considering the added rules. Let's take the example in Fig. 8 to do explanation. It has three inserted rules, i.e., $rule_{1-3}$.

Algorithm 4. Ratio Adjustment

Input: FK constraint c_i and the added rules R

```

1:  $p_T \leftarrow 0, p_F \leftarrow 0$ 
2: for all rule  $r \in R$  do
3:    $\eta \leftarrow \text{condition of } r, p_\eta \leftarrow 1$ 
4:   for all status  $s \in \eta$  do
5:      $p_\eta \leftarrow p_\eta \times p_s$ 
6:   end for
7:    $p_\eta \leftarrow p_\eta \times p_{j\text{filter}}$ 
8:   if target status of  $r$  is  $T$  then
9:      $p_T \leftarrow p_T + p_\eta$ 
10:  else
11:     $p_F \leftarrow p_F + p_\eta$ 
12:  end if
13: end for
14:  $p'_T \leftarrow c_i.p_T - p_T, p'_F \leftarrow c_i.p_F - p_F$ 
15:  $c_i.\text{ratio} \leftarrow p'_T / (p'_T + p'_F)$ 

```

First, it initializes two counters (line 1), i.e., p_T and p_F , for counting the occupied cumulative probabilities of the statuses T and F which are forcibly generated by the rules. Then, it iterates over all rules within FK constraints (line 2), e.g., $rule_1: [FT \leftarrow T]$. For each rule r , it calculates (lines 3-6) the occurrence probability p_η of its pre-condition η (e.g., $\eta = T$ in $rule_1$), which is the occupied cumulative probability of the target status ζ indicated by the leftmost bit of the rule (e.g., $\zeta = F$ in $rule_1$). p_η is calculated on the joinable probabilities of all the involved bit statuses, i.e., p_s (line 5). For any status in η , e.g., F of $\eta = FT$ in $rule_2: FFT \leftarrow FT$, if it is forcibly generated by a previous rule, e.g., the 2nd F completely determined by $rule_1$, its p_s is set to 1; otherwise p_s is set to the remaining cumulative probability of the involved FK constraints, e.g., $\eta = T$ in $rule_1: FT \leftarrow T$, its p_s is set to 0.3. Notice that the rules are effective only if all the involved FILTER constraints can be satisfied, which is measured by the joint probability of these FILTER constraints, i.e., $p_{j\text{filter}}$ (line 7). Supposing there are 10% keys satisfying both $Filter_1$ and $Filter_2$ in Fig. 8, if $\eta = T$, $p_\eta = 0.3 * 0.1 = 0.03$ in $rule_1$. After obtaining p_η for the condition T in $rule_1$, we update the corresponding counter for its target status ($\zeta = F$) of the rule (lines 8-11), i.e., $p_F = 0 + 0.03 = 0.03$. Next, when all rules have been iterated, the algorithm calculates (line 14) the remaining cumulative probabilities p'_T and p'_F for statuses T and F of current FK constraint c_i , where $c_i.p_T$ and $c_i.p_F$ are the original cumulative probabilities of the statuses T and F without rules. For example, for c_2 of $Chain_b$, $p'_T = c_2.p_T - 0.6 * 0.4 = 0.24$ and $p'_F = c_2.p_F - 0.03 = 0.4 * 0.4 - 0.03 = 0.13$. Finally, the ratio of current FK constraint $c_i.\text{ratio}$

is adjusted according to the remaining cumulative probabilities by $\frac{p'_T}{p'_T + p'_F}$, e.g., $c_2.\text{ratio} = \frac{0.24}{0.24 + 0.13} \approx 0.65$. For a more detailed example of the ratio adjustment is explained in Appendix B, available in the online supplemental material.

For a FK constraint c_i , if the adjusted ratio is invalid, that is not between 0 and 1, the adjustment for constraint c_i is illegal and the introduced error is defined as $e_i = |\text{Min}\{p'_T, p'_F, 0\}| \times s^T$, where s^T is the size of the involved table. Therefore, the error caused by all illegal adjustments is $e = \sum_{1 \leq i \leq k} e_i$, where k is the number of FK constraints.

6 RELATED WORK

A amount of work has contributed to the generation of synthetic databases in the last decades. Depending on whether queries are considered during the generation, we can classify them into two types, i.e., schema-aware data generators and query-aware ones.

Schema-aware data generators [1], [4], [7], [12], [13], [14], [15], [23], [26] aim to simulate the target database on data characteristics, e.g., data distribution. Alexander *et al.* [4] proposes pseudo-random number generators to realize the parallel data generation. Torlak [26] supports a scalable data generation by a class of multi-dimensional models. RSGen [23] reverses the database metadata, especially the statistics, to generate a synthetic DBI (database instance) with similar data characteristics. Work in [15] uses search-based techniques to ensure that the generated data meets the original integrity constraints. However, all these data generators cannot guarantee to generate test databases satisfying the specified workload characteristics, i.e., cardinality size, of an application.

Query-aware data generators aim to generate DBIs satisfying the constraints defined along with the queries. They can be used for either functional testing or performance evaluation. For functional testing, it expects to generate a DBI meeting the testing purpose, e.g., maximize *coverage*. Predicate coverage is a popular criterion used to guide the test database generation while satisfying the validation of queries [16], [24], [28]. The difficulty is how to generate a small database which should be efficient for correctness verification and be diverse enough to reveal faults in queries. In work [27], it presents an approach to generate a reducing test database to meet the same coverage and similar error detection capabilities by analyzing the queries and the initial database. In addition, work in [8], [9], [11], [17] tries to generate queries to maximize the code coverage by systematically exploring a programmer's execution paths, i.e., interactions between queries and the database. These work focuses on how to guarantee the test target based on the (generated) DBI and the queries, which is essential for guaranteeing the correctness of databases.

There is the other branch of query-aware data generators, which is designed for database performance evaluation with respect to a specific application [5], [6], [19], [20], [22], among which [6], [19], [20] are a series of work. For an application, its query templates and the intermediate cardinality sizes along the query trees are given as the inputs for the database generation. The purpose is to generate one DBI and instantiate the parameters in query templates, which is expected to have the same execution performance, e.g.,

latency, as the one from the production environment. QAGen [6] is the first query-aware data generator for performance testing, but for each query it generates an individual DBI and its CSP (constraint satisfaction program) has the usability limitations. WAGen [19] makes a great improvement that it generates $m (\leq n)$ DBIs with n input queries, but WAGen cannot guarantee to generate only one DBI. Though MyBenchmark [20] has done a lot for optimization, generating one DBI is still difficult for multiple queries. Moreover, for integrating multiple symbolic databases generated by QAGen corresponding to input queries, huge constrained bipartite graphs (CBG) are constructed by [19], [20], which consume much memory, and the algorithm based on CBGs is difficult to parallelize. DCGen [5] and HYDRA [22] use novel methods to represent data distribution with ideas from the probabilistic graphical model. But DCGen is weak in supporting foreign key constraints, and it cannot easily support parallel data generation. HYDRA [22] still has huge computation complexity in generation, which limits its ability in processing complex queries, and it has low data randomness but with a lot of duplicate data. Considering the problem mentioned, our work aims to provide a scalable way to generate one DBI and instantiate the query templates to simulate the production environment with respect to a specific application for performance evaluation instead of functional testing.

7 EXPERIMENTS

Considering query-aware data generation for performance evaluation, we conduct experiments on *Touchstone* to answer the following questions.

RQ1: How does *Touchstone* perform against the state-of-the-art query-aware data generator?

RQ2: How efficient and effective is *Touchstone* in generating a simulated DBI?

RQ3: What is the influence of introducing the outer join algorithm?

Environment. Our experiments are conducted on a cluster with 8 nodes connected using 1 Gigabit Ethernet. Each node is equipped with 2 Intel Xeon E5-2620 @ 2.0 GHz CPUs, 64GB memory and 3 TB HDD disk configured in RAID-5.

Workloads. SSB [21] is a benchmark designed to measure query performance for analytical applications, and its tables are organized with a star schema. We include all its 13 queries, i.e., $Q_{1.1-1.3}$, $Q_{2.1-2.3}$, $Q_{3.1-3.4}$ and $Q_{4.1-4.3}$, divided into 4 groups, i.e., Q_{1-4} . TPC-H [3] is the representative decision support benchmark. Because of its complexity in workloads, we include the first 16 queries among all 22 queries, i.e., Q_{1-16} , for *Touchstone*. To the best of our knowledge, *Touchstone* provides the widest support to TPC-H workload among all the related studies [5], [6], [19], [20]. A synthesized micro benchmark is used to illustrate the ability supporting outer joins.

Comparison. We compare *Touchstone* with the state-of-the-art work MyBenchmark² [20]. Workloads for comparison is selected from TPC-H including $Q_{2,3,6,10,14,16}$, since the other

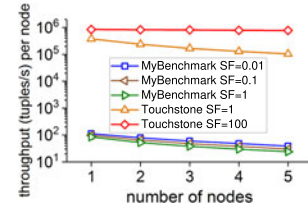


Fig. 9. Comparison of data generation throughput.

queries will degrade the performance of MyBenchmark significantly and not be used for comparison.

Input Generation. To build valid inputs for experiments, we generate the DBI and queries of TPC-H and SSB using tools *dbgen* and *qgen*.³ DBI is imported into MySQL database and its schema is used as the input H . We can obtain the input data characteristics D for all columns for the DBI in MySQL. Running benchmark queries on the DBI, we can get their physical query plans from MySQL query parser and optimizer, and then identify cardinality constraints for operators. Input workload characteristics W are constructed by the parameterized queries and the cardinality constraints. Note that, we can generate DBIs with different scale factors (SF) using the same input W by employing selectivities instead of the absolute cardinalities in input constraints.

Settings. The error threshold (desired precision) and maximal iterations in query instantiation are set to 10^{-4} and 20 respectively. The default maximum allowed size L in join information table is set to 10^4 .

7.1 Comparison With MyBenchmark

To answer *RQ1*, we compare *Touchstone* with MyBenchmark from four aspects, including data generation throughput, scalability to multiple nodes, memory consumption and processing capability of complex workloads.

Fig. 9 shows the data generation throughputs per node of *Touchstone* and MyBenchmark as we vary the number of nodes under different SFs . Due to the unacceptably long processing time of MyBenchmark, we adopt smaller SFs for it and larger SFs for *Touchstone*. Overall, the data generation throughput of *Touchstone* is at least 3 orders of magnitude higher than that of MyBenchmark. This is because MyBenchmark does not have a good parallelization or an efficient data generation schema. Furthermore, as the number of nodes increases, the data generation throughput per node of MyBenchmark decreases dramatically for all three SFs . Although the decline of generation throughput per node of *Touchstone* is obvious when $SF = 1$, *Touchstone* is linearly scalable when $SF = 100$. This is because for a small target database the distributed maintenance rather than data generation dominates the computational cost in *Touchstone*, while its overhead comparatively diminishes by increasing the size of the target database.

Fig. 10 reports the peak memory consumptions of *Touchstone* and MyBenchmark under different data scales. The memory usage of MyBenchmark mainly consists of two parts, namely, memory consumed by MyBenchmark Tool and by PostgreSQL for managing intermediate states. The

2. We would like to thank Eric Lo for providing source code of MyBenchmark.

3. Tools for TPC-H and SSB: <http://www.tpc.org/tpch/>; <https://github.com/electrum/ssb-dbgen>.

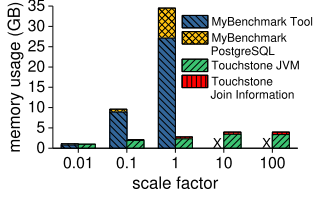


Fig. 10. Comparison of memory consumption.

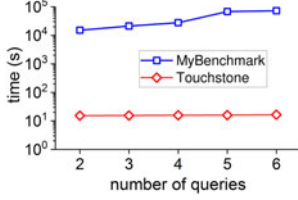


Fig. 11. Comparison of data generation time.

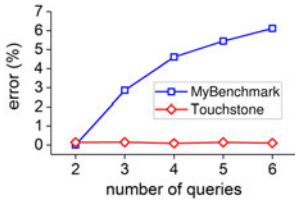


Fig. 12. Comparison of global relative error.

memory usage of *Touchstone* mainly includes memory for JVM itself and for maintaining join information. The memory consumption of *Touchstone* is much lower than that of MyBenchmark under the same *SFs*. This is because the symbolic databases and constrained bipartite graphs in MyBenchmark are very large and consume a large amount of memory, but the join information table in *Touchstone* has been effectively compressed. It is worth noting that the memory consumption of *Touchstone* remains almost constant when *SF* > 10. This is because for *Touchstone*, the JVM itself occupies most of the memory, while the join information maintenance only occupies a tiny piece of memory.

Figs. 11 and 12 present data generation time and global relative error separately of *Touchstone* and MyBenchmark as we vary the number of input queries with *SF*=1. The queries are loaded in order of their serial numbers. In Fig. 11, the data generation time of *Touchstone* grows very little when the number of queries increases, significantly outperforming MyBenchmark. In Fig. 12, the error of *Touchstone* is much smaller than that of MyBenchmark. Moreover, as there are more input queries, the global relative error of *Touchstone* remains small, while the error of MyBenchmark has an obvious rise.

It can be seen that MyBenchmark can not be easily applied to generate the large scale of database for complex workloads. In the following, we focus on demonstrating the advantages of *Touchstone* using more workloads.

7.2 Evaluating TouchStone on TPC-H and SSB

To answer *RQ2*, we evaluate the impact of workload complexity on query instantiation, data generation, the generation

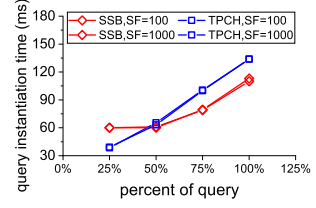


Fig. 13. Query instantiation time.

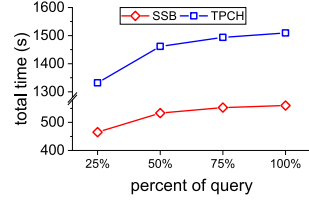


Fig. 14. Data generation time.

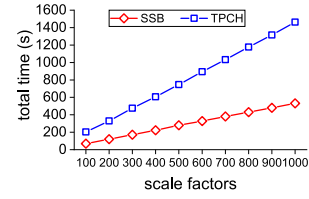


Fig. 15. Scalability to data scales.

scalability as well as the generation fidelity and the status for join of *Touchstone* on both TPC-H and SSB.

7.2.1 Generation Efficiency

Fig. 13 shows the query instantiation time of *Touchstone* as we vary the number of queries with different *SFs*. The query instantiator is deployed on a single node. When all 16 queries in TPC-H and all 13 queries in SSB are used as inputs, query instantiation completes within 0.2s. There is little difference in instantiation time for *SF*=100 and *SF*=1000, as the complexity of query instantiation is independent of data scales but only correlated with the complexity of input workloads.

Fig. 14 shows the data generation time of *Touchstone* as we vary the number of queries with *SF* = 1000. *Touchstone* is deployed on 8 nodes. We can see the generation time increases slowly as the number of queries increases for both TPC-H and SSB. For *Q*₇₋₈ in TPC-H, there are relatively more cardinality constraints over equi-join operators, so the time increment is larger when we change from 6 queries to 8 queries. But when the number of queries increases from 10 to 16, the time increment is almost indiscernible, for *Q*₁₁₋₁₆ in TPC-H are simple, among which *Q*₁₂₋₁₅ have no cardinality constraints on equi-join operators.⁴ SSB has less complex workload templates, and it is faster to generate the database. Fig. 15 presents the data generation time of *Touchstone* under different *SFs* with the input of all queries from both SSB and TPC-H. Because the generation of each tuple is independent, which need not be stored in memory, the

4. Depending on the physical query plans of *Q*₁₂₋₁₅ in TPC-H, all foreign keys must be joined for the equi-join operators and the sizes of output tuples are determined.

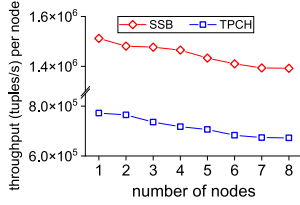


Fig. 16. Scalability to multiple nodes.

TABLE 1
Relative Errors for TPC-H and SSB

	TPC-H		SSB	
	$error_G$	$error_{max}$	$error_G$	$error_{max}$
SF = 1	0.19%	3.14%	0.50%	1.99%
SF = 2	0.17%	3.65%	0.63%	2.9%
SF = 3	0.16%	2.76%	0.37%	1.27%
SF = 4	0.15%	3.26%	0.95%	2.2%
SF = 5	0.14%	2.60%	0.94%	2.94%

data generation throughput of *Touchstone* is stable for different data scales. It costs about 140s (resp. 50s) for every 100G data generation with TPC-H (resp. SSB) workloads and the control cost for distributed data generation is almost stable, i.e., 60s (resp. 20s) for TPC-H (resp. SSB) workloads. Moreover, when $SF = 1000$, the total runtime of *Touchstone* is less than 25 minutes for TPC-H (9 minutes for SSB).

Fig. 16 presents the data generation throughput per node of *Touchstone* as we vary the number of nodes with $SF = 1000$. The input includes all the queries. The distributed controller is deployed on one node, and then we continually add nodes to deploy the data generator. When it starts parallel data generation, it has a synchronization requirement among nodes, which causes a moderate degradation in generation on each node. To the best of our knowledge, *Touchstone* is the first query-aware data generator for application simulation, which can support full parallel data generation on multiple nodes. So it is capable of supporting industrial scale database generation.

7.2.2 Data Fidelity

The data fidelity of the synthetic DBI is evaluated by the relative error on cardinality constraints and the performance deviation on query latencies. We calculate the relative error for each query in the similar way with global relative error, i.e., $error_G$, which only involves the cardinality constraints in each individual query. The global (resp. individual) relative deviation of latencies is defined

$$\frac{\sum_{q_i \in Q} |q_{i,l} - \hat{l}_i|}{\sum_{q_i \in Q} q_{i,l}} \quad (\text{resp. } \frac{|q_{i,l} - \hat{l}_i|}{q_{i,l}})$$

with $q_{i,l}$ and $\hat{l}_i$ as the expected and the actual latencies for q_i . We compare the latency of query processing on the database generated by *dbgen* against that on the synthetic DBI generated by *Touchstone* to show the performance deviation.

Table 1 shows the relative errors for TPC-H Q_{1-16} and SSB with different SFs . Since the cardinalities of each intermediate result in the query tree need to be counted by the database, and MySQL is very slow when importing large sized tables, we run this group of experiments with SFs no more than 5. For TPC-H, the maximum error $error_{max}$ among all 16 queries is less than 4% and the global relative

TABLE 2
Performance Deviation of Query Latency

	TPC-H		SSB	
	dev_{avg}	dev_{max}	dev_{avg}	dev_{max}
SF = 1	3.60%	8.8%	1.83%	5.3%
SF = 2	3.50%	8.9%	0.99%	4.0%
SF = 3	3.12%	6.6%	2.31%	7.5%
SF = 4	3.91%	9.27%	1.51%	3.9%
SF = 5	3.60%	8.66%	2.07%	6.1%

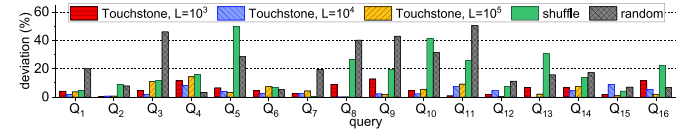


Fig. 17. Performance deviation of TPC-H.

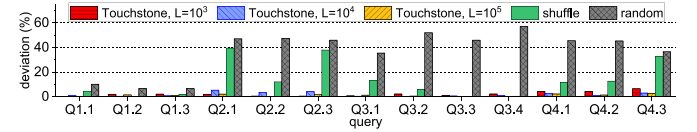


Fig. 18. Performance deviation of SSB.

error $error_G$ is less than 0.2% for all SFs ; for SSB, $error_{max}$ and $error_G$ are less than 3% and 1% for all SFs .

Table 2 presents the average and maximum performance deviations on latencies of TPC-H and SSB queries with $SF = 1-5$, i.e., dev_{avg} and dev_{max} . For TPC-H (resp. SSB), the maximum and the average performance deviations are 9.27% (resp. 7.5%) and 3.91% (resp. 2.31%). We can see that the performance deviation is inconspicuous for all queries.

In Figs. 17 and 18, we investigate the fidelity for each individual query with different data generation methods for TPC-H and SSB by using different L , i.e., 10^3 , 10^4 and 10^5 , for *Touchstone*. We compare the latencies by different methods against the one generated by *dbgen* with $SF = 1$. Here, *shuffle* means to shuffle the original data generated by *dbgen* as done in Google [10]; *random* means to populate random values into the database. Specifically, *shuffle* is consistent in data distribution of the original data and we use the original queries; *random* is generated by a schema-aware data generator with the inputs as the database schema, data characteristics for all non-key columns and the reference distributions between foreign keys and primary keys, and the parameters in query templates are assigned randomly. From the results, we can see that 1) *Touchstone* has the least deviations compared to *shuffle* and *random*, and it is not sensitive to L ; 2) neither *random* nor *shuffle* can guarantee the original table dependencies, and then they have worse simulation fidelity; 3) *shuffle* is highly sensitive to the complex join operations, e.g., Q_5 in TPC-H.

7.2.3 Insight into FK Constraint Adjustments

In Algorithm 3, we adjust the FK constraints by adding rules to avoid the appearance of mismatch cases. In Figs. 19 and 20, we present the maximum number of entries in the join information table and the maximum number of rules

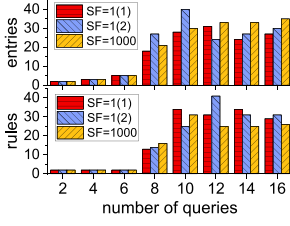


Fig. 19. The max numbers of entries and rules of TPC-H.

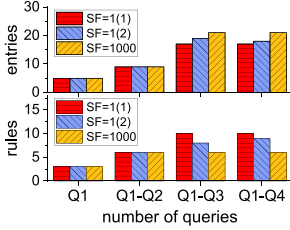


Fig. 20. The max numbers of entries and rules of SSB.

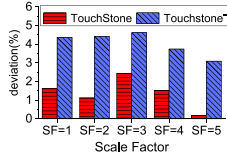


Fig. 21. Comparison of relative deviation for latency of Q13.

added in the FK constraints during data generation as we vary the number of queries with $SF=1$ and $SF=1000$ for TPC-H and SSB. The input queries are loaded in order of their serial numbers. For $SF=1$, there are two groups of experimental results, represented by $SF=1(1)$ and $SF=1(2)$. We can see that 1) the maximum numbers of entries and rules are randomly generated by comparing $SF=1(1)$ and $SF=1(2)$, which are dependent on the order of constraint chains and the result of query instantiation; 2) the numbers of entries and rules increase obviously for the first 10 queries in TPC-H. Because the target (or related) foreign key is within table *Order* of TPC-H, and there is no cardinality constraint over the equi-join operator associated with this foreign key in Q_{11-16} , which is consistent with the result shown in Fig. 14. Overall, the maximum numbers of entries and rules are both small, which are less than 45 for both TPC-H and SSB.

In conclusion, the result strongly illustrates the correctness of our work, validates the necessity of query-aware data generation and L in *Touchstone* has no significant influence on fidelity.

7.3 Outer Join Algorithm Evaluation

To answer *RQ3*, we compare the performance of current version of *Touchstone* with the one in the conference paper without outer join processing, called *Touchstone⁻* to demonstrate the influence of outer joins.

7.3.1 Left Outer Join in TPC-H

Since SSB has no outer joins, we run the experiment on Q_{13} in TPC-H which has an outer join. Since *null*-tuples generated by the outer join of Q_{13} are much less than the tuples generated by its inner joins, i.e., about one twenty-fifth, it then

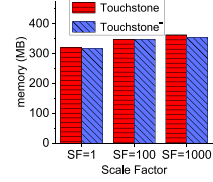


Fig. 22. Comparison of memory consumption.

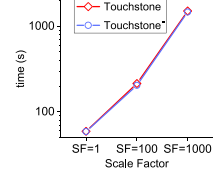


Fig. 23. Comparison of generation time.

Table	attribute	type	cardinality	aveLen	key
Part	Partid	int	200K	-	1
	Supplier	string	100	6	0
	Type	string	100	6	0
	Stock	int	10K	-	0
	Procid	int	800K	-	1
Product	Pr_Partid	int	200K	-	0
	Kind	string	100	6	0
	Sales Volume	int	1M	-	0

q1: SELECT Type, SUM (SalesVolume) FROM Part LEFT JOIN Product ON P_Partid=Pr_Partid GROUP BY Type;
q2: SELECT Kind, COUNT (*) FROM Part RIGHT JOIN Product ON P_Partid=Pr_Partid WHERE Type = ? GROUP BY Kind;
q3: SELECT P_Partid, Productid FROM Part FULL OUT JOIN Product ON P_Partid=Pr_Partid WHERE Supplier = ? AND SalesVolume > ?;
q4: SELECT Productid FROM Part, Product ON P_Partid=Pr_Partid WHERE Stock < ?;

Fig. 24. Example workloads of outer joins.

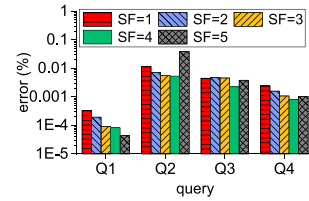


Fig. 25. Comparison of relative error.

affects the relative/global error little and we can get an excellent fidelity even if we take the inner join method to simulate this outer join. But *TouchStone* has better accuracy in simulating outer joins exposed by the deviation of latencies, as shown in Fig. 21, which lowers the relative deviation by 2% for all *SF*'s compared to *TouchStone⁻*. Furthermore, we take a single-node generation mode to show the memory overhead and generation efficiency in Figs. 22 and 23. By an asynchronous join table caching, *Touchstone* can guarantee memory usage and generation efficiency comparable to *Touchstone⁻*.

7.3.2 Outer Join Evaluation on Synthetic Workloads

We extract a set of workloads from real-world application to demonstrate the support to outer join operators as shown in Fig. 24. It constructs four types of workloads, i.e., q_1 to q_4 , among which q_1 is a left outer join query, q_2 is a right outer join query, q_3 is a full outer join query and q_4 is an inner-join query. Detailed discussion can be found in Appendix C, available in the online supplemental material. And then we run the synthetic workloads on the tables and collect the real results and latency.

Figs. 25 and 26 show the relative error on result sizes and process latencies for $SF=1$ to 5 by *Touchstone*. We can see that all relative errors are below 0.1% and the latencies are exactly the same as the real ones. Overall, *Touchstone* can support outer joins well.

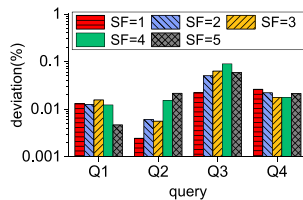


Fig. 26. Comparison of relative deviation for latency.

We can see that *Touchstone* can improve the simulation fidelity with less relative errors or deviations but with no additional requirements on resources.

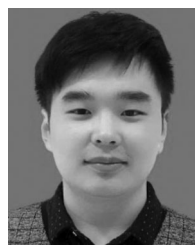
8 DISCUSSION AND CONCLUSION

In this paper, we introduce *Touchstone*, a query-aware data generator with characteristics of completely parallelizable and bounded usage to memory. And *Touchstone* is linearly scalable to computing resource and data scales. It is the first work supporting well the data generation on non-equi-joins and outer-joins considering the fixed cardinality constraints in workloads. *Touchstone* has demonstrated it is the most efficient data generator among the state-of-art work, and the generated DBI is consistent with expectations in both cardinalities and query latencies. In order to make our work repeatable, we make the source code and running examples of this paper available in github [2].

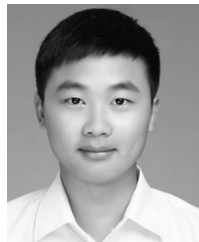
However *Touchstone* is not general enough to support all scenarios. It does not support filters on key columns with the assumption that keys are identifiers of tuples and generally have no physical meaning. Equality constraints over filters involving multiple columns are not supported, which are very strict constraints, and have not been found in workloads of standard benchmarks. Similar to related work [5], [29], [30], Equi-joins on columns with no reference constraint are not supported in our work. *Touchstone* requires the primary-foreign-key dependencies must form a direct acyclic graph (DAG), which does not support the database schema with cyclic reference relationship. We are working on the solution targeting these limitations.

REFERENCES

- [1] DTM data generator, 1998. [Online]. Available: <http://www.sqledit.com/dg/>
- [2] Technical report, running examples and source code of *Touchstone*, 2019. [Online]. Available: <https://github.com/dbhammer/Touchstone>
- [3] TPC-H benchmark, 1993. [Online]. Available: <http://www.tpc.org/tpch/>
- [4] A. Alexandrov, K. Tzoumas, and V. Markl, "Myriad: Scalable and expressive data generation," *Proc. VLDB Endowment*, vol. 5, no. 12, pp. 1890–1893, 2012.
- [5] A. Arasu, R. Kaushik, and J. Li, "Data generation using declarative constraints," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2011, pp. 685–696.
- [6] C. Binnig, D. Kossmann, E. Lo, and M. T. Özsu, "QAGen: Generating query-aware test databases," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2007, pp. 341–352.
- [7] N. Bruno and S. Chaudhuri, "Flexible database generators," in *Proc. 31st Int. Conf. Very Large Data Bases*, 2005, pp. 1097–1107.
- [8] D. Chays, Y. Deng, P. G. Frankl, S. Dan, F. I. Vokolos, and E. J. Weyuker, "An AGENDA for testing relational database applications," *Softw. Testing Verification Rel.*, vol. 14, no. 1, pp. 17–44, 2004.
- [9] D. Chays, J. Shahid, and P. G. Frankl, "Query-based test generation for database applications," in *Proc. 1st Int. Workshop Testing Database Syst.*, 2008, pp. 1–6.
- [10] S. Deep, A. Gruenheid, K. Nagaraj, H. Naito, J. Naughton, and S. Viglas, "DIAMetrics: Benchmarking query engines at scale," *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 3285–3298, 2020.
- [11] M. Emmi, R. Majumdar, and K. Sen, "Dynamic test input generation for database applications," in *Proc. Int. Symp. Softw. Testing Anal.*, 2007, pp. 151–162.
- [12] J. Gray, P. Sundaresan, S. Englert, K. Baclawski, and P. J. Weinberger, "Quickly generating billion-record synthetic databases," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 1994, pp. 243–252.
- [13] J. E. Hoag and C. W. Thompson, "A parallel general-purpose synthetic data generator," *ACM SIGMOD Rec.*, vol. 36, no. 1, pp. 19–24, 2007.
- [14] K. Houkjaer, K. Torp, and R. Wind, "Simple and realistic data generation," in *Proc. 32nd Int. Conf. Very Large Data Bases*, 2006, pp. 1243–1246.
- [15] G. M. Kapfhammer, P. McMinn, and C. J. Wright, "Search-based testing of relational schema integrity constraints across multiple database management systems," in *Proc. IEEE 6th Int. Conf. Softw. Testing Verification Valid.*, 2013, pp. 31–40.
- [16] S. A. Khalek, B. Elkarablieh, Y. O. Laleye, and S. Khurshid, "Query-aware test generation using a relational constraint solver," in *Proc. 23rd IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2008, pp. 238–247.
- [17] C. Li and C. Csallner, "Dynamic symbolic database application testing," in *Proc. 3rd Int. Workshop Testing Database Syst.*, 2010, Art. no. 7.
- [18] Y. Li, R. Zhang, X. Yang, Z. Zhang, and A. Zhou, "Touchstone: Generating enormous query-aware test databases," in *Proc. USENIX Conf. Usenix Annu. Tech. Conf.*, 2018, pp. 575–586.
- [19] E. Lo, N. Cheng, and W.-K. Hon, "Generating databases for query workloads," *Proc. VLDB Endowment*, vol. 3, no. 1/2, pp. 848–859, 2010.
- [20] E. Lo, N. Cheng, W. W. Lin, W.-K. Hon, and B. Choi, "MyBenchmark: Generating databases for query workloads," *VLDB J.*, vol. 23, no. 6, pp. 895–913, 2014.
- [21] P. O'Neil, E. O'Neil, X. Chen, and S. Revilak, "The star schema benchmark and augmented fact table indexing," in *Proc. Technol. Conf. Perform. Eval. Benchmarking*, 2009, pp. 237–252.
- [22] A. Sanghi, R. Sood, J. R. Haritsa, and S. Tirthapura, "Scalable and dynamic regeneration of big data volumes," in *Proc. 21st Int. Conf. Extending Database Technol.*, 2018, pp. 301–312.
- [23] E. Shen and L. Antova, "Reversing statistics for scalable test databases generation," in *Proc. 6th Int. Workshop Testing Database Syst.*, 2013, pp. 1–6.
- [24] M. J. Suárez-Cabal, C. de la Riva, J. Tuya, and R. Blanco, "Incremental test data generation for database queries," *Automated Softw. Eng.*, vol. 24, no. 4, pp. 719–755, 2017.
- [25] T. Syrjänen, "Logic programs and cardinality constraints: Theory and practice." Ph.D. thesis, Helsinki Univ. Technol., 2009.
- [26] E. Torlak, "Scalable test data generation from multidimensional models," in *Proc. ACM SIGSOFT 20th Int. Symp. Found. Softw. Eng.*, 2012, pp. 1–11.
- [27] J. Tuya, C. de la Riva, M. J. Suarez-Cabal, and R. Blanco, "Coverage-aware test database reduction," *IEEE Trans. Softw. Eng.*, vol. 42, no. 10, pp. 941–959, Oct. 2016.
- [28] J. Tuya, M. J. Suárez-Cabal, and C. De La Riva, "Full predicate coverage for testing SQL database queries," *Softw. Testing Verification Rel.*, vol. 20, no. 3, pp. 237–288, 2010.
- [29] E. Zamanian, C. Binnig, and A. Salama, "Locality-aware partitioning in parallel database systems," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2015, pp. 17–30.
- [30] J. Zhang and Y. Tay, "DScaler: Synthetically scaling a given relational database," *Proc. VLDB Endowment*, vol. 9, no. 14, pp. 1671–1682, 2016.



Qingshuai Wang received the BS degree in software engineering in 2019 from East China Normal University, where he is currently working toward the PhD degree. His current research interests include transaction processing, data management, and database benchmarking.



Yuming Li received the PhD degree from East China Normal University in 2019. He is currently with Hua Wei Ltd. His current research interests include transaction processing and data management.



Zhenjie Zhang received the PhD degree from the National University of Singapore in 2010. He is currently the head of data in Neuron Mobility. He was a research scientist and senior research scientist with the Advanced Digital Sciences Center, University of Illinois. He is the program committee member of top AI, machine learning, and database conferences.



Rong Zhang received the PhD degree in computer science from Fudan University in 2007. Since 2011, she has been joined East China Normal University and is currently a professor with the University. Her current research interests include distributed data management and database benchmarking. She is a member of China Computer Federation.



Aoying Zhou is currently a professor in computer science with East China Normal University. He is the vice-director of the ACM SIGMOD China and Database Technology Committee of China Computer Federation. His research interests include data management, transaction processing, and database benchmarking.



Ke Shu graduated from the Institute of Computing Technology, Chinese Academy of Sciences. At PingCAP, his work is focused on database testing, and his research interests include distributed system testing, automated testing, database workload generation, and chaos engineering.

▷ **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.**